Network Programming Exam Study Guide

Process Control · Signals · Pipes · FIFOs · Mutexes & Condition Variables — built directly from Stevens, *Advanced Programming in the UNIX Environment*

Contents

How to use this guide **3**

PART 1 — Process Control (27/7): fork, exec, zombies, inheritance, fd behavior **3**
- 1.1 Process concepts recap . . . 3
- 1.2 `fork()` — APUE Ch. 8 . . . 3
- 1.3 The classic APUE synchronization idiom: `TELL_WAIT` / `TELL_PARENT` / `TELL_CHILD` / `WAIT_PARENT` / `WAIT_CHILD` . . . 4
- 1.4 The `exec` family — APUE Ch. 8 . . . 6
- 1.5 File sharing internals — the three-table kernel model (Stevens Ch. 3, this is a favorite exact-diagram question) . . . 7
- 1.6 Zombies & process termination . . . 8
- 1.7 File descriptor behavior — summary table . . . 8

PART 2 — Signals (29/7, 3/8) **8**
- 2.1 What a signal is . . . 8
- 2.2 Registering handlers . . . 9
- 2.3 Reliable vs. unreliable signals (a real Stevens Ch. 10 topic, often glossed over) . . . 9
- 2.4 Common signals to know cold . . . 9
- 2.5 Signal masks & blocking . . . 9
- 2.6 Async-signal-safety . . . 10
- 2.7 Signals tying back into process control . . . 10
- 2.8 `kill()` and `raise()` . . . 10
- 2.9 `alarm()` and `pause()` . . . 10
- 2.10 Interrupted system calls (`EINTR`) — Stevens §10.5, very commonly tested . . . 11
- 2.11 Reentrant / async-signal-safe functions — the concrete APUE table . . . 12
- 2.12 `sigsuspend()` — atomically unblock-and-wait . . . 12
- 2.13 `SIGCLD` vs. `SIGCHLD` (a historical quirk worth knowing) . . . 12

PART 3 — Pipes (5/8, 10/8) **13**
- 3.1 `pipe()` — APUE Ch. 15 . . . 13
- 3.2 Code walkthrough — `pipe.c` . . . 13
- 3.3 The universal pipe idiom: close what you don't need . . . 14
- 3.4 Answers to the pipe lab experiments . . . 14
- 3.5 Pipe capacity & blocking behavior . . . 14

PART 4 — FIFOs (10/8) **14**
- 4.1 FIFO vs. pipe . . . 14
- 4.2 Code walkthrough — `fifo.c` . . . 15

4.3 Answers to the FIFO lab experiments 16

PART 5 — Mutexes & Condition Variables (12/8, 17/8) 17
5.1 The problem: race conditions 17
5.2 Mutexes 17
5.3 Condition variables 17
5.4 Producer–consumer pattern 18

PART 6 — Cross-cutting comparison table 18

PART 7 — "Small points" checklist: easy-to-forget Stevens details 18

PART 8 — Predict-the-output practice 19

PART 9 — 74 Likely Exam Questions 22
Process control / fork / zombies 22
exec family 23
Signals 23
Pipes 24
FIFOs 24
Mutexes / condition variables 24

How to use this guide

Your professor draws directly from Stevens' APUE, and she may hand you a code snippet and ask what it prints. That means two separate skills are being tested:

1. **Theory recall** — precise definitions, what's copied vs. shared, why races happen, exact function signatures.
2. **Output prediction** — given a piece of code, trace through it (accounting for non-determinism) and say what would print, or what the *range* of valid outputs is.

This guide covers both. Part 8 at the end is dedicated entirely to "predict the output" practice, since that's the "different story" skill you flagged.

PART 1 — Process Control (27/7): fork, exec, zombies, inheritance, fd behavior

1.1 Process concepts recap

- A process = an executing instance of a program: text (code), data (globals), heap, stack, plus OS-maintained state: PID, PPID, open file descriptor table, signal dispositions, current working directory, environment, resource limits.
- `pid_t getpid(void)` — own PID. `pid_t getppid(void)` — parent's PID.
- Every process except `init` (PID 1) or its nearest subreaper has a parent.

1.2 `fork()` — APUE Ch. 8

```c
#include <unistd.h>
pid_t fork(void);
```

Return value — the single most-tested fact: - Returns **0** in the child. - Returns the **child's PID** in the parent. - Returns **-1** on failure (no child created), sets `errno` (`EAGAIN`, `ENOMEM`).

What is copied vs. shared — Stevens is explicit about this list:

Copied (child gets its own independent copy): - Process address space: text, data, heap, stack (copy-on-write in practice, but semantically independent from this point on) - Process ID (new) - Parent process ID (= the forking process's PID) - File descriptor *table* (the array of fd's itself — but see below, entries still point to shared kernel objects) - Pending alarms are **cleared** in the child - The set of pending signals for the child is **cleared** (set to the empty set)

Shared / inherited as-is: - Effective/real/saved user & group IDs - Environment variables - Controlling terminal - Current working directory, root directory, `umask` - Open file descriptors — and critically, **the underlying open file table entries are shared**, not duplicated (see §1.5) - Signal dispositions/handlers (the *registration*, not pending signals) - Record locks are **not** inherited by the child (APUE explicitly calls this out)

Order of execution after fork() is undefined — Stevens states this directly ("we never know whether the child starts executing before the parent, or vice versa"). This is exactly the race behind lab experiment 1.

Stevens' own worked example: fork() + unflushed stdio (a classic "small point" that is very testable)

APUE Ch. 8's canonical example writes with `write()` *before* `fork()`, then calls `printf()` in both parent and child. The result differs depending on whether stdout is connected to a terminal or redirected to a file:

- **Interactive (terminal):** stdout is **line buffered**, so the `printf()` before `fork()` is flushed (because of the trailing `\n`) before the fork happens → you see the line **once**.
- **Redirected to a file:** stdout becomes **fully buffered**, so the pre-fork `printf()` output sits in the stdio buffer, **unflushed**, at the moment of `fork()`. Since the child gets a **copy of the parent's memory** (including that unflushed stdio buffer), **both** parent and child eventually flush their own copy of the buffer when they exit → the line is printed **twice**.

This is a very high-value "small point": *stdio buffering state is part of process memory, and `fork()` duplicates whatever is sitting in the buffer at that instant — including data not yet written to the actual file.* The fix is to `fflush(stdout)` (or `fflush(NULL)` for all streams) immediately before calling `fork()` if you don't want doubled output.

ANSI C buffering rules (Ch. 5 — know these exactly)

1. Standard input and standard output are **fully buffered**, unless they refer to an interactive device, in which case they are **line buffered**.
2. **Standard error is never fully buffered** — traditionally unbuffered by default (some modern libc implementations make it line-buffered, but APUE's baseline rule is: `stderr` is unbuffered so error messages appear immediately).
3. Line-buffered streams flush when a newline is written, when the buffer fills, or when input is requested from an unbuffered stream / a line-buffered stream that needs data from the kernel.
4. You can change this with `setbuf()` / `setvbuf()`.

1.3 The classic APUE synchronization idiom: `TELL_WAIT` / `TELL_PARENT` / `TELL_CHILD` / `WAIT_PARENT` / `WAIT_CHILD`

Because fork() order is a race, if the parent and child *each* need to finish some setup step before the other proceeds, you cannot just assume an ordering. Stevens' `apue.h` header provides a signal-based rendezvous:

```c
#include "apue.h"
TELL_WAIT();                /* call once, before fork(), sets up signal handling */

if ((pid = fork()) < 0) {
    err_sys("fork error");
} else if (pid == 0) {          /* child */
    /* child does its setup ... */
    TELL_PARENT(getppid());     /* tell parent: I'm done */
    WAIT_PARENT();               /* wait until parent tells me it's done */
    /* child continues ... */
} else {                         /* parent */
    /* parent does its setup ... */
    TELL_CHILD(pid);
    WAIT_CHILD();
    /* parent continues ... */
}
```

This is implemented with `SIGUSR1`/`SIGUSR2` signals under the hood (in the classic APUE implementation) — it's the textbook answer to "how do you deterministically synchronize a parent and child instead of just hoping the scheduler behaves." If your professor asks "how would you fix the race in `myfork.c` so output is always in the same order," **this is the canonical Stevens answer** — not just "add a sleep" (sleeps are a hack, not a guarantee).

Code walkthrough — `myfork.c`

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>

int global = 10;

int main(int argc, char **argv)
{
    pid_t   pid;

    if ((pid = fork()) == 0) {
        printf("In Child Process: ");
        global += 5;
        sleep(5);
    } else if (pid > 0) {
        printf("In Parent Process: ");
        global += 10;
    } else {
        fprintf(stderr, "Error in fork\n");
        exit(1);
    }

    printf("pid = %d, ppid = %d\n", getpid(), getppid());
    printf("Global = %d\n", global);
    exit(0);
}
```

- `global` is a file-scope variable initialized *before* `fork()`. After `fork()`, parent and child each hold an **independent copy**; incrementing it in one never affects the other's copy.
- Note that here, unlike Stevens' doubled-output example, each branch calls `printf()` *after* the fork with a trailing `\n` — so on a terminal each process's own output is flushed on its own, and you will not see doubling; the ordering between the two processes' lines is still a race, though.

Answers to the `myfork.c` lab experiments

1. **Order of print statements, multiple runs:** Non-deterministic — depends on OS scheduling of parent vs. child after `fork()`. Different runs, and different machines, can show parent-first or child-first.
2. **Sleep 5s in both, check with `ps`:** While both sleep, `ps -ef` shows two processes; the child's `PPID` equals the parent's `PID`.
3. **Remove sleep in child only:** The child finishes quickly; if the parent hasn't yet called `wait()`, the child becomes a **zombie** (`Z` / `<defunct>` in `ps`) until reaped.
4. **Remove sleep in parent only:** The parent exits almost immediately; the child is **re-parented to `init`** (or the nearest subreaper). `getppid()` in the child then returns 1 (or

the subreaper's PID).
5. **Global + local values, no modification:** Identical in both — full memory copy at the instant of `fork()`.
6. **Modify separately:** Each process only sees its own modification — proves address spaces are independent, not just "copied once."
7. **Write "From Parent"/"From Child" to a shared fd opened pre-fork, no `\n`:** Because the fd (and its file offset) is **shared** via the same open-file-table entry, writes append sequentially without overwriting, but the *order* is a race, and unbuffered small writes may appear merged on one line without newlines.
8. **Add `\n`:** Cleaner line separation; the sequential-append (no-overwrite) guarantee is unchanged — only formatting differs.

1.4 The `exec` family — APUE Ch. 8

```c
#include <unistd.h>
int execl(const char *path, const char *arg0, ..., (char *)0);
int execv(const char *path, char *const argv[]);
int execle(const char *path, const char *arg0, ..., (char *)0, char *const envp[]);
int execve(const char *path, char *const argv[], char *const envp[]);
int execlp(const char *file, const char *arg0, ..., (char *)0);
int execvp(const char *file, char *const argv[]);
```

Naming convention: - **l** vs **v** — argument list given inline (l) vs. as an array (v) - **p** — search `PATH` to resolve a bare filename - **e** — caller supplies its own `envp[]`; otherwise the current environment (`environ`) is used

Only **`execve()`** is the actual kernel system call; the other five are library functions layered on top of it (Stevens draws this exact relationship diagram in Ch. 8 — know that `execve` is the one true syscall).

Critical facts: - `exec*()` replaces the calling process's text/data/heap/stack. **The PID does not change.** - On success, `exec*()` **never returns** — there's no "rest of the old program" left. - On failure, returns **-1**, sets `errno` (`ENOENT`, `EACCES`, `ENOEXEC`…). - **Open file descriptors survive `exec()` by default**, unless `FD_CLOEXEC` was set via `fcntl(fd, F_SETFD, FD_CLOEXEC)`. - Signal handlers set to a **function** are reset to `SIG_DFL` (the handler code no longer exists in the new image); handlers set to `SIG_IGN` **remain** `SIG_IGN`. - Real UID/GID are preserved; **effective** UID/GID change if the new program has the set-user-ID/set-group-ID bit set (a whole exam topic on its own: saved set-user-ID).

Code walkthrough — `myexec.c`

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char **argv)
{
    pid_t   pid;
    int     rv;

    if ((pid = fork()) < 0) {
        fprintf(stderr, "Error forking\n");
```

```
        exit(1);
    } else if (pid == 0) {
        rv = execl("hello", "hello", NULL);
        if (rv < 0) {
            fprintf(stderr, "Error execing\n");
            exit(2);
        }
    } else {
        wait(NULL);
        printf("Exiting Parent...\n");
    }
    exit(0);
}
```

The "check `rv < 0`" idiom only ever fires on **failure** — on success there's nothing left of the old program to check it.

Answers to the exec lab experiments

1. **Call `execl()` directly, no fork:**
   - **(a) Known command:** succeeds, replaces the current image; code after `execl()` **never runs**.
   - **(b) Unknown/invalid command:** fails, returns `-1`, `errno` set (typically `ENOENT`); the **original program keeps running**, so the error-handling code after `execl()` does execute.
2. **exec in parent vs. child:**
   - **exec in child (idiomatic):** parent's image untouched, keeps running, can `wait()`.
   - **exec in parent:** the parent's own image is replaced (loses its own code/state, PID unchanged), while the child (still the original program) runs independently — inverts the usual roles.
3. **Open fd pre-fork, pass as string to `execl()`, write in child after exec without re-opening: Works**, because fd's survive `exec()` (absent `FD_CLOEXEC`) — the child `atoi()`s the string back to an int and calls `write(fd, ...)` directly.

1.5 File sharing internals — the three-table kernel model (Stevens Ch. 3, this is a favorite exact-diagram question)

Stevens describes **three kernel data structures** that together implement file sharing:

1. **Per-process file descriptor table** — one per process; each entry holds: (a) file descriptor flags (just `FD_CLOEXEC`), and (b) a **pointer** to an entry in the file table.
2. **System-wide (kernel) file table** — shared across all processes; each entry holds: the **file status flags** (read, write, append, sync, nonblocking, set by `open()`), the **current file offset**, and a **pointer** to the entry's v-node table entry.
3. **V-node (i-node) table** — one entry per actual open file; holds the v-node information: file type, operations pointers, and (for regular files) the i-node — file size, owner, permissions, etc.

The exam-critical consequences: - If **two independent `open()` calls** open the same file (even in the same process), you get **two separate file table entries** — each with its **own file offset** — both pointing to the same v-node. - If a **single fd is duplicated** (`dup()`, `dup2()`) — or **shared via `fork()`** — the copies point to the **same file table entry**, so they **share the file offset and file status flags**. This is exactly why parent and child, sharing a pre-fork fd, never overwrite each other when both write: they're advancing the *same* offset counter in

the *shared* file table entry. - After `fork()`: the child's fd table is a **copy** of the parent's (new table, same fd numbers), but each copied entry **still points to the same file table entry** as the parent's — this is the precise mechanism behind "shared open file description" that most students only know informally.

1.6 Zombies & process termination

- Termination paths: `exit()`, `_exit()`/`_Exit()`, return from `main()`, or an uncaught signal.
  - `exit()` — flushes/closes stdio streams, runs `atexit()`-registered functions (**in LIFO order** — a specific Stevens detail), then calls `_exit()`.
  - `_exit()` — terminates immediately at the kernel level, no stdio flushing.
- A terminated process whose parent hasn't yet retrieved its exit status is a **zombie** — the kernel retains its PID and exit status/resource stats until `wait()`/`waitpid()` reaps it.

```
#include <sys/wait.h>
pid_t wait(int *status);
pid_t waitpid(pid_t pid, int *status, int options);
```

- `wait()` blocks until *any* child terminates.
- `waitpid()` targets a specific PID (or `-1` for any, matching `wait()`); `WNOHANG` makes it non-blocking (poll).
- Status-decoding macros: `WIFEXITED(status)`, `WEXITSTATUS(status)`, `WIFSIGNALED(status)`, `WTERMSIG(status)`, `WIFSTOPPED(status)`.

Orphans: if a parent dies first, the child is reparented to `init` (PID 1) or the nearest `PR_SET_CHILD_SUBREAPER`. `init` calls `wait()` periodically specifically to reap its adopted orphans.

Zombie prevention techniques: 1. Call `wait()`/`waitpid()` after the child is expected to finish. 2. Catch `SIGCHLD`, and inside the handler loop `waitpid(-1, NULL, WNOHANG)` until it returns ≤ 0. 3. Explicitly set `signal(SIGCHLD, SIG_IGN)` — on POSIX-compliant systems this causes children to be auto-reaped, never becoming zombies. 4. **Double fork**: parent forks a child; child immediately forks a grandchild and `exit()`s. The grandchild is reparented to `init`, which reaps it eventually — the original parent only ever briefly `wait()`s for its short-lived direct child. (This is also literally in APUE Ch. 8 as the illustrated "orphaned process" example.)

1.7 File descriptor behavior — summary table

Across `fork()`	Across `exec()`
fd **table** is copied (independent per process)	fd table **preserved as-is** (same numbers stay open)
but copies **point to the same file table entry** → shared offset, shared status flags	fd's stay open **unless** `FD_CLOEXEC` is set
closing an fd in one process doesn't affect the other	—

PART 2 — Signals (29/7, 3/8)

2.1 What a signal is

An asynchronous notification — from the kernel (`SIGSEGV`, `SIGFPE`), another process (`kill()`), or self (`raise()`, `alarm()` → `SIGALRM`).

2.2 Registering handlers

```c
#include <signal.h>
void (*signal(int signum, void (*handler)(int)))(int);   /* legacy interface */
int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);
```

- `signal()`'s semantics **historically varied across UNIX flavors** (some reset the handler to `SIG_DFL` after one delivery — "unreliable signals" in old System V) — this is precisely *why* Stevens pushes `sigaction()` as the **reliable**, POSIX-standard interface.
- `sigaction()` lets you specify: the handler, a mask of signals to block **while the handler runs** (`sa_mask`), and flags (`sa_flags`), notably `SA_RESTART` (auto-restart interrupted slow system calls) and `SA_SIGINFO` (richer handler signature with `siginfo_t`).
- A handler is `SIG_DFL` (default), `SIG_IGN` (ignore), or a function pointer.

2.3 Reliable vs. unreliable signals (a real Stevens Ch. 10 topic, often glossed over)

- **Unreliable signals** (old-style): the signal disposition was reset to default right before the handler ran, creating a race where a second occurrence of the same signal (arriving before you re-armed the handler) would use the default action instead of your handler — this is literally why early UNIX signal handling was called "unreliable."
- **Reliable signals** (POSIX, via `sigaction`): the disposition does *not* reset automatically; instead, delivery of additional instances of the *same* signal is automatically **blocked while its own handler is running** (unless you deliberately unblock it), preventing that race.
- Signals are **not queued** — while a given signal type is blocked/pending, multiple additional occurrences of it are typically collapsed into one pending delivery (contrast with real-time signals, which *can* queue — out of scope here but worth knowing exists).

2.4 Common signals to know cold

Signal	Default action	Typical cause
`SIGCHLD`	ignored	child terminated or stopped
`SIGPIPE`	terminate	write to a pipe/socket with no reader
`SIGALRM`	terminate	`alarm()` timer expired
`SIGINT`	terminate	Ctrl-C
`SIGKILL`	terminate	cannot be caught, blocked, or ignored
`SIGSTOP`	stop	cannot be caught, blocked, or ignored
`SIGTERM`	terminate	polite termination request (default of `kill`)
`SIGUSR1/SIGUSR2`	terminate	user-defined (used internally by `TELL_WAIT` family)

2.5 Signal masks & blocking

```c
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

- `SIG_BLOCK`, `SIG_UNBLOCK`, `SIG_SETMASK`.

- Blocking ≠ ignoring: a **blocked** signal is held **pending** and delivered once unblocked; an **ignored** signal's occurrence is discarded outright, never delivered.
- `sigpending()` lets you inspect which signals are currently blocked-and-pending.

2.6 Async-signal-safety

A handler can interrupt the program at *any* instruction, including mid-`malloc()`. Only a defined, small set of "async-signal-safe" functions (e.g., `write()`, `_exit()`) are safe inside a handler; calling non-reentrant functions like `printf()` or `malloc()` inside a handler risks corrupting internal library state (classic exam gotcha).

2.7 Signals tying back into process control

- `SIGCHLD` + `sigaction` + a `waitpid(-1, NULL, WNOHANG)` loop is the textbook "reliable" zombie-reaping design — asynchronous, doesn't block your main flow.
- Writing to a pipe with no readers → `SIGPIPE` (default: terminate) and/or `write()` returns `-1`/`EPIPE` if `SIGPIPE` is handled/ignored.

2.8 `kill()` and `raise()`

```c
#include <signal.h>
int kill(pid_t pid, int signo);
int raise(int signo);           /* equivalent to kill(getpid(), signo) */
```

`kill()`'s `pid` argument has **four distinct cases** (a favorite exact-rules question): `- pid > 0` — send `signo` to the specific process with that PID. `- pid == 0` — send to **every process in the same process group** as the caller. `- pid < 0` (and `!= -1`) — send to every process whose **process group ID equals `|pid|`**. `- pid == -1` — send to **every process** the caller has permission to signal (implementation-dependent, typically all processes except `init` and the caller itself).

2.9 `alarm()` and `pause()`

```c
#include <unistd.h>
unsigned int alarm(unsigned int seconds);
int pause(void);
```

- `alarm()` schedules `SIGALRM` to be delivered after the given number of seconds; a value of `0` **cancels** any pending alarm. It returns the number of seconds **remaining** on any previously set (and now replaced/cancelled) alarm — a classic "what does this return" trap.
- `pause()` **suspends the calling process until any signal is caught** by a handler (or a signal whose default action is to terminate/stop the process occurs). It's the building block for a "sleep until something happens" pattern.

The classic APUE `alarm()` + `pause()` race, and why it needs fixing

A naive way to implement a timed sleep:

```c
unsigned int sleep1(unsigned int seconds)
{
    if (signal(SIGALRM, sig_alrm) == SIG_ERR)
        return seconds;
    alarm(seconds);
```

```
    pause();
    return alarm(0);
}
```

The race: if `SIGALRM` fires (and the handler returns) *before* the process reaches `pause()`, the `pause()` call has **already missed** the signal and blocks **forever** (or until some unrelated signal arrives). This is a textbook "the signal arrived in the gap between two lines of code" race — exactly the same category of bug as the fork-ordering race, just for signals instead of processes.

Stevens' fix, using `sigsetjmp/siglongjmp`:

```
#include <setjmp.h>
#include <signal.h>
#include <unistd.h>

static jmp_buf env_alrm;

static void sig_alrm(int signo) { longjmp(env_alrm, 1); }

unsigned int sleep2(unsigned int seconds)
{
    if (signal(SIGALRM, sig_alrm) == SIG_ERR)
        return seconds;
    if (setjmp(env_alrm) == 0) {
        alarm(seconds);
        pause();
    }
    return alarm(0);
}
```

The handler jumps out via `longjmp()` instead of just returning, so even if the alarm fires before `pause()` is reached, control returns to right after `setjmp()` — closing the race window. (The fully robust version additionally uses `sigprocmask`/`sigsuspend` to also handle the case where `SIGALRM` interrupts a *different* signal handler; this is APUE §10.19 territory if your professor wants the complete version.)

2.10 Interrupted system calls (`EINTR`) — Stevens §10.5, very commonly tested

- **Historically**, if a process caught a signal while blocked in a **"slow" system call**, the call was interrupted, returned `-1`, and set `errno = EINTR` — the assumption being that since something happened worth catching, the blocked call should probably wake up and let you decide.
- **"Slow" system calls** are ones that can block **indefinitely**: `read()`/`write()` on pipes, terminals, and some network devices; `open()` on some device files (or a FIFO, as in Part 4); `wait()`/`waitpid()`; and a few others. Ordinary disk-file I/O is generally *not* considered "slow" since it completes in bounded time.
- **The standard defensive idiom:**

```
again:
    if ((n = read(fd, buf, BUFFSIZE)) < 0) {
        if (errno == EINTR)
            goto again;         /* just an interrupted system call — retry */
```

```
        /* handle other real errors */
    }
```

- **`SA_RESTART`** (a `sigaction` flag) tells the kernel to **automatically restart** certain interrupted slow system calls itself, rather than returning `EINTR` at all — sparing you from writing the retry loop for that signal. Note it's a **per-signal** setting, and not every syscall honors it identically (e.g., a `read()` that already transferred some data before being interrupted returns that partial count rather than restarting).

2.11 Reentrant / async-signal-safe functions — the concrete APUE table

A signal handler can interrupt the main program at *any* point — including in the middle of a non-reentrant library call that was, say, updating a shared static data structure (like `malloc()`'s internal free list). If the handler then calls that same non-reentrant function again, internal state can be corrupted.

- **Async-signal-safe** (safe to call inside a handler): `_exit()`, `write()`, `read()`, `signal()`, most of the `sig*` family itself, `kill()`, `alarm()`, and a defined POSIX list of similarly low-level calls.
- **Not safe** inside a handler (non-reentrant / may use static buffers or global lock state): `printf()`/`fprintf()` family, `malloc()`/`free()`, `exit()` (as opposed to `_exit()`), most stdio functions in general.
- **Practical pattern:** a handler should do the **minimum possible** — typically just set a `volatile sig_atomic_t` flag, or `write()` a byte to a pipe (the "self-pipe trick," used to safely wake up a `select()`/`poll()` loop from within a handler) — and let the main program act on that outside the handler.

2.12 `sigsuspend()` — atomically unblock-and-wait

```
#include <signal.h>
int sigsuspend(const sigset_t *sigmask);
```

- Temporarily replaces the process's signal mask with `sigmask`, then **suspends the process until a signal is caught**, then **restores the original mask** before returning. The "replace mask + block" step is **atomic**, which is exactly what makes it useful for closing races like: "block a signal, check/modify some shared state depending on it, then atomically unblock-and-wait for it" without a window where the signal could sneak in between the unblock and the wait.

2.13 `SIGCLD` vs. `SIGCHLD` (a historical quirk worth knowing)

Older System V UNIX used `SIGCLD` with unusual, non-portable semantics (e.g., setting its disposition to `SIG_IGN` could change how zombies were handled in surprising ways, and its default action differed across implementations). POSIX standardized on **`SIGCHLD`** with the well-defined semantics described in §2.4/§1.6. If your professor mentions `SIGCLD`, she's almost certainly testing whether you know it's the old, non-portable name/behavior that `SIGCHLD` replaced.

PART 3 — Pipes (5/8, 10/8)

3.1 `pipe()` — APUE Ch. 15

```c
#include <unistd.h>
int pipe(int fd[2]);
```

- Creates an **unnamed, half-duplex** channel. `fd[0]` = read end, `fd[1]` = write end.
- Returns 0 on success, -1 on failure.
- Only useful between **related** processes (fd's must be inherited via `fork()`), since there's no filesystem name.

3.2 Code walkthrough — `pipe.c`

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <unistd.h>
#include <wait.h>

#define MAX 256
#define STDOUT 1

int main(int argc, char **argv)
{
    pid_t   pid;
    int     fd[2];

    if (pipe(fd) < 0) {
        fprintf(stderr, "Error opening pipe\n");
        exit(2);
    }

    if ((pid = fork()) == 0) {
        printf("In Child Process...fd[0] = %d, fd[1] = %d\n", fd[0], fd[1]);
    }
    else if (pid > 0) {
        printf("In Parent Process(child pid = %d), fd[0] = %d, fd[1] = %d\n",
                pid, fd[0], fd[1]);
        wait(NULL);
    }
    else {
        fprintf(stderr, "Error in fork\n");
        exit(1);
    }

    printf("Exiting from process %d\n", getpid());
    exit(0);
}
```

`pipe()` must be called before `fork()` — only then do parent and child inherit fd's referring to the *same* kernel pipe. Calling `pipe()` after `fork()` gives each process its own unrelated pipe,

useless for IPC between them.

3.3 The universal pipe idiom: close what you don't need

For parent → child communication: parent closes `fd[0]` and writes to `fd[1]`; child closes `fd[1]` and reads from `fd[0]`.

Why this matters (frequently tested): - If a reader keeps its own write end open, `read()` on an empty pipe **blocks forever**, because the kernel still thinks a writer *could* write — EOF (`read()` returning 0) is only signaled once **all** write-end fd's referencing the pipe are closed. - If the intended reader closes its read end while a writer keeps writing, the writer receives `SIGPIPE`/`EPIPE`.

3.4 Answers to the pipe lab experiments

1. **Unidirectional parent→child:** parent closes `fd[0]`, writes; child closes `fd[1]`, reads and prints.
2. **Two pipes for bidirectional communication:** `pipe(fd1)` (parent→child) and `pipe(fd2)` (child→parent) before `fork()`. Parent closes `fd1[0]` and `fd2[1]`; writes via `fd1[1]`, reads via `fd2[0]`. Child closes `fd1[1]` and `fd2[0]`; reads via `fd1[0]`, writes via `fd2[1]`.
3. **One writes-then-reads, other reads-then-echoes:** classic **ping-pong** — process A writes, then blocks reading the echo; process B blocks reading first, then immediately writes back what it received, unblocking A. Complementary operations avoid deadlock.
4. **Both read first: Deadlock** — neither ever reaches its `write()`, since each is blocked waiting for the other.
5. **Child closes both fds then sleeps 5s; parent sleeps 2s then writes:** Parent's `write()` to a pipe with **no reader at all** (and no possible future reader) triggers **`SIGPIPE`** (default: terminate the parent) and/or `write()` returns `-1`/`EPIPE` if `SIGPIPE` is handled/ignored.

3.5 Pipe capacity & blocking behavior

- Finite kernel buffer (historically 4 KB; commonly 64 KB on modern Linux, adjustable via `fcntl(fd, F_SETPIPE_SZ, ...)`).
- `write()` blocks once the buffer is full (until drained); `read()` blocks on an empty pipe until data arrives or all write ends close (→ EOF, `read()` returns 0).
- Writes up to `PIPE_BUF` bytes are **guaranteed atomic** — won't interleave with another process's concurrent write to the same pipe. `PIPE_BUF` is queryable via `fpathconf(fd, _PC_PIPE_BUF)` and is typically 4096 bytes on Linux.

PART 4 — FIFOs (10/8)

4.1 FIFO vs. pipe

	Pipe	FIFO
Created by	`pipe()`	`mkfifo()`
Named in the filesystem?	No	**Yes**
Usable between unrelated processes?	No	**Yes** — any process with path access

	Pipe	FIFO
Persists after processes exit?	No	Directory entry persists until `unlink()`ed; still no stored *data*

```c
#include <sys/types.h>
#include <sys/stat.h>
int mkfifo(const char *path, mode_t mode);
```

4.2 Code walkthrough — `fifo.c`

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <errno.h>

#define FIFO1 "/tmp/fifo.1"
#define FIFO2 "/tmp/fifo.2"
#define FILE_MODE (S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH)

void client(int, int), server(int, int);

int main(int argc, char **argv)
{
    int readfd, writefd;
    pid_t childpid;

    if ((mkfifo(FIFO1, FILE_MODE) < 0) && (errno != EEXIST)) {
        fprintf(stderr, "can't create %s\n", FIFO1);
        exit(1);
    }
    if ((mkfifo(FIFO2, FILE_MODE) < 0) && (errno != EEXIST)) {
        unlink(FIFO1);
        fprintf(stderr, "Can't create %s\n", FIFO2);
        exit(2);
    }

    if ((childpid = fork()) == 0) {
        printf("In child\n");
        if ((readfd = open(FIFO1, O_RDONLY, 0)) < 0) {
            fprintf(stderr, "Error opening FIFO %s(%s)\n", FIFO1, strerror(errno));
            exit(3);
        }
        writefd = open(FIFO2, O_WRONLY, 0);
        server(readfd, writefd);
        exit(0);
```

```
    }

    printf("In parent\n");
    writefd = open(FIFO1, O_WRONLY, 0);
    readfd  = open(FIFO2, O_RDONLY, 0);
    client(readfd, writefd);

    waitpid(childpid, NULL, 0);
    close(readfd);
    close(writefd);
    unlink(FIFO1);
    unlink(FIFO2);
    exit(0);
}
```

- `mkfifo()` deliberately tolerates `EEXIST` so reruns don't fail just because the FIFO already exists.
- **Rendezvous / blocking-open behavior (heavily tested):** by default, `open()` on a FIFO for `O_RDONLY` **blocks until some other process opens it for `O_WRONLY`**, and vice versa. The child opens `FIFO1` read-first, then `FIFO2` write-first; the parent opens `FIFO1` write-first, then `FIFO2` read-first — complementary, so both eventually unblock.
- `O_NONBLOCK` changes this: a **non-blocking** `open()` for `O_RDONLY` succeeds immediately even with no writer yet; a **non-blocking** `open()` for `O_WRONLY` with no reader present **fails** with `ENXIO`. (Worth knowing as the "what if we pass O_NONBLOCK" follow-up question.)

4.3 Answers to the FIFO lab experiments

1. **Sleep in parent, check files exist:** Yes — `mkfifo()` creates a real directory entry (type `p` in `ls -l`), visible regardless of whether any process currently has it open.
2. **After exit, with `unlink()` present:** No longer exist — explicit `unlink()` removes the directory entries.
3. **Comment out `unlink()`:** FIFOs **persist** in `/tmp` — a FIFO's name is an ordinary filesystem entry, independent of any process holding it open.
4. **Reboot, no `unlink()`:** On Linux `/tmp` is commonly tmpfs/cleared at boot, so the FIFO typically **doesn't survive** — but that's a filesystem-configuration fact, not a FIFO property; on a **persistent local filesystem** (e.g., your laptop's home directory, ext4/APFS) it **does** survive a reboot as metadata. On **NFS**, FIFOs are **not reliably usable for actual IPC** across clients — POSIX doesn't guarantee the open()/read()/write() rendezvous semantics hold over NFS (a known Stevens/UNP caveat).
5. **Remove with `rm`?** Yes — a FIFO is just a special file type; `rm`/`unlink()` removes its directory entry like any file.
6. **Reverse the open order (both read-first on the same FIFO): Deadlock** — both processes block in `open()` waiting for a counterpart that isn't coming in the expected order. Lesson: a read-open and a write-open on the same FIFO must always be issued by *different* processes without both sides waiting on each other first.

PART 5 — Mutexes & Condition Variables (12/8, 17/8)

5.1 The problem: race conditions

Multiple threads (or processes with shared memory) accessing shared data concurrently, unsynchronized, can interleave arbitrarily → a **race condition**. The code region touching shared data is the **critical section**.

5.2 Mutexes

```c
#include <pthread.h>
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_trylock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

- Only **one thread at a time** can hold a mutex. `pthread_mutex_lock()` blocks if another thread holds it; `pthread_mutex_trylock()` returns immediately with `EBUSY` instead of blocking.
- Lock immediately before, unlock immediately after, the critical section — holding too long hurts concurrency; too little (or skipping entirely) reintroduces the race.
- Deadlock risk from inconsistent lock-acquisition order across threads; fix by always acquiring locks in a **consistent global order**.

5.3 Condition variables

```c
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);
int pthread_cond_signal(pthread_cond_t *cond);
int pthread_cond_broadcast(pthread_cond_t *cond);
```

- Lets a thread **sleep until notified**, without busy-waiting.
- `pthread_cond_wait()` **atomically** unlocks the mutex and blocks; on wakeup it **re-acquires** the mutex before returning — this atomicity closes the gap between "check condition" and "go to sleep," during which a signal could otherwise be missed.
- **Always call `cond_wait()` in a `while` loop re-checking the predicate**, never an `if`:

  ```c
  pthread_mutex_lock(&mutex);
  while (!condition_is_true) {
      pthread_cond_wait(&cond, &mutex);
  }
  /* critical section */
  pthread_mutex_unlock(&mutex);
  ```

  Guards against **spurious wakeups** (the OS may wake a waiter with no real signal) and multiple waiters racing over the same signaled condition.
- `pthread_cond_signal()` wakes **at least one** waiter; `pthread_cond_broadcast()` wakes **all** of them.

5.4 Producer–consumer pattern

- Shared bounded buffer + `pthread_mutex_t lock` + two condition variables: `not_full`, `not_empty`.
- Producer: lock → while full, `cond_wait(not_full, lock)` → insert → `cond_signal(not_empty)` → unlock.
- Consumer: lock → while empty, `cond_wait(not_empty, lock)` → remove → `cond_signal(not_full)` → unlock.
- Same underlying idea as pipe blocking (Part 3), but with explicit synchronization primitives instead of a kernel-managed buffer.

PART 6 — Cross-cutting comparison table

Mechanism	Related processes only?	Named/persistent?	Byte-stream or message?	Synchronizes shared *memory* access?
`fork()` shared fd	Yes (parent + descendants)	No	n/a	n/a
Pipe	Yes	No	Byte stream, half-duplex	No
FIFO	**No**	**Yes**	Byte stream, half-duplex	No
Mutex	n/a	No	n/a	**Yes** — mutual exclusion
Condition variable	n/a	No	n/a	**Yes** — blocking/waking on a predicate

---

PART 7 — "Small points" checklist: easy-to-forget Stevens details

Use this as a last-pass sanity check the night before:

- `fork()` clears **pending signals** and **pending alarms** in the child, but **not** the signal handler *registrations* — those are inherited.
- Record locks (`fcntl` locks) are **not** inherited across `fork()`.
- `exec()` preserves `SIG_IGN` dispositions but resets caught-signal dispositions to `SIG_DFL` (the handler code is gone).
- `_exit()` skips stdio flushing; `exit()` does not.
- `atexit()`-registered functions run in **LIFO** order (last registered, first run).
- Two independent `open()` calls on the same file → **two different file table entries**, each with its **own** offset — even though both point to the same v-node/i-node.
- `dup()`/`dup2()` and `fork()`-inherited fd's **share** the file table entry (and thus offset) — the opposite situation from the point above.
- Unflushed stdio buffer contents at the moment of `fork()` get **duplicated into the child's memory** — this is the mechanism behind the classic "doubled output when redirected to a file" gotcha.

- `SIGCHLD`'s default disposition is to be **ignored** (not terminate) — but that doesn't stop zombies from forming; ignoring the *default* disposition and explicitly setting `SIG_IGN` are two different things (the explicit `SIG_IGN` is what triggers auto-reaping on POSIX systems).
- Old-style (`signal()`) handling could silently revert to `SIG_DFL` after one delivery on some historical systems — this "unreliable signal" behavior is exactly why `sigaction()` is preferred.
- `PIPE_BUF`-sized (or smaller) writes to a pipe are atomic; larger writes can be interleaved with another writer's data.
- A FIFO's `open()` call for read blocks until a writer opens it, **and vice versa** — this rendezvous is bypassed only with `O_NONBLOCK` (with different, asymmetric behavior for the read-side vs. write-side non-blocking open).
- `pthread_cond_wait()`'s mutex unlock+block is **atomic** specifically to prevent a "lost wakeup" race — this is the single most common short-answer question on condition variables.

PART 8 — Predict-the-output practice

For each snippet, reason about (a) what is guaranteed, and (b) what varies by scheduling. Answers follow each snippet.

8.1

```c
int x = 1;
if (fork() == 0) {
    x++;
    printf("child x=%d\n", x);
} else {
    x += 10;
    printf("parent x=%d\n", x);
    wait(NULL);
}
```

Answer: Always `child x=2` and `parent x=11` — regardless of which line prints first (that ordering is a race), because `x` is duplicated at `fork()` and each process only ever modifies its own copy. The two lines can appear in **either order**, but their *content* is fixed.

8.2

```c
int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
write(fd, "A", 1);
if (fork() == 0) {
    write(fd, "B", 1);
} else {
    wait(NULL);
    write(fd, "C", 1);
}
```

Answer: File will always contain `"A"` followed by `"B"` and `"C"` in **some order** (either `ABC` or `ACB`), never with either overwriting the other and never both landing at offset 1 — because the fd (and its offset) is shared via the same file table entry, and the parent explicitly `wait()`s

before its own write, guaranteeing the child's `write()` happens first in this particular case, so the deterministic answer is **`ABC`**.

8.3

```
printf("Hello");         /* no \n, output redirected to a file: ./a.out > out.txt */
fork();
```

Answer: `out.txt` will contain `HelloHello` — stdout is fully buffered when redirected to a file, so `"Hello"` sits unflushed in the stdio buffer at `fork()` time; both parent and child inherit a copy of that buffer and each flushes it independently at exit.

8.4

```
printf("Hello\n");        /* run interactively on a terminal, no redirection */
fork();
```

Answer: Only **one** `Hello` — on a terminal stdout is line-buffered, and the trailing `\n` flushes it *before* `fork()` runs, so there's nothing left in the buffer to duplicate.

8.5

```
pid_t pid = fork();
if (pid == 0) {
    exit(5);
}
int status;
wait(&status);
if (WIFEXITED(status))
    printf("child exited with %d\n", WEXITSTATUS(status));
```

Answer: Deterministically prints `child exited with 5` — `wait()` blocks until the child exits, so there's no race on *this* outcome (only on how fast it happens), and `WEXITSTATUS` correctly extracts the value passed to `exit()`.

8.6

```
int fd[2];
pipe(fd);
if (fork() == 0) {
    close(fd[0]);
    write(fd[1], "hi", 2);
    exit(0);
} else {
    close(fd[1]);
    char buf[10];
    int n = read(fd[0], buf, 10);
    printf("read %d bytes\n", n);
}
```

Answer: Deterministically `read 2 bytes` — the parent's `read()` blocks until the child writes "hi" (2 bytes) and then either returns those 2 bytes or blocks for more depending on timing; since the child then exits (closing its write end) and no more data is coming, this specific single

`read()` call returns as soon as data is available, which is exactly the 2 bytes written — order of write-then-read is enforced by blocking, not by scheduling luck.

8.7

```c
int fd[2];
pipe(fd);
close(fd[1]);          /* close write end immediately, no fork */
char buf[10];
int n = read(fd[0], buf, 10);
printf("n = %d\n", n);
```

Answer: `n = 0` — with **no** write end open anywhere, `read()` on the pipe returns 0 immediately (EOF), rather than blocking forever; this is the "all write ends closed" EOF condition explained in Part 3.

8.8

```c
int fd[2];
pipe(fd);
/* write end never closed by anyone */
char buf[10];
int n = read(fd[0], buf, 10);  /* single process, no fork */
printf("n = %d\n", n);
```

Answer: This **hangs forever** (blocks) — the write end is still open (in this same process even), so the kernel keeps waiting for possible data; nothing ever gets printed. Common trick question: forgetting to `fork()`/write anything, or forgetting to close the write end, produces a silent deadlock instead of an EOF.

8.9

```c
signal(SIGCHLD, SIG_IGN);
if (fork() == 0) {
    exit(0);
}
sleep(1);
printf("checked\n");
/* Question: does a zombie appear in `ps` during the 1-second sleep? */
```

Answer: **No** — explicitly setting `SIGCHLD` to `SIG_IGN` causes the kernel (on POSIX-compliant systems, including Linux) to auto-reap terminated children immediately, so no zombie ever appears, even transiently.

8.10

```c
if (fork() == 0) {
    execl("/bin/echo", "echo", "child", NULL);
    printf("exec failed\n");    /* only reached if execl fails */
    exit(1);
}
wait(NULL);
printf("parent done\n");
```

Answer: Deterministic content, race on interleaving with the child's own output: the child's process image becomes `/bin/echo`, which prints `child\n` to stdout and exits — `"exec failed"` is **never** printed since `/bin/echo` exists and `execl` succeeds. The parent's `wait(NULL)` blocks until the child (now running as `echo`) finishes, so `parent done` is guaranteed to print **after** `child`.

8.11

```
unsigned int r = alarm(10);
r = alarm(5);
printf("r = %u\n", r);
```

Answer: `r = 10` (approximately — could print 9 depending on timing) — `alarm()` returns the number of seconds **remaining** on any previously scheduled alarm that this call replaces; the second call cancels the ~10-second-remaining alarm and starts a new 5-second one.

8.12

```
/* signal(SIGALRM, handler) installed WITHOUT SA_RESTART */
alarm(2);
int n = read(fd, buf, 100);   /* fd is a pipe, no data arriving within 2s */
printf("n = %d, errno==EINTR? %d\n", n, errno == EINTR);
```

Answer: `n = -1, errno == EINTR` (prints `1`) — `read()` was blocked on a "slow" device (pipe) with no data; `SIGALRM` fired after 2s, the handler ran and returned, and since `SA_RESTART` was **not** set, the interrupted slow system call returns `-1/EINTR` instead of resuming.

PART 9 — 74 Likely Exam Questions

Process control / fork / zombies

1. What does `fork()` return in the parent, in the child, and on error?
2. What exactly is copied vs. shared between parent and child after `fork()`?
3. Why is the order of parent/child `printf()` output after `fork()` non-deterministic?
4. Why do parent and child, sharing the same fd after fork(), also share the same *file offset*?
5. What is copy-on-write, and why does the kernel use it for `fork()`?
6. Define a zombie process. Why does it exist instead of the kernel just discarding it?
7. What is an orphan process, and who adopts it?
8. Name three distinct techniques to prevent zombie accumulation.
9. Difference between `wait()` and `waitpid()`? When would you use `WNOHANG`?
10. What happens to pending signals and alarms across `fork()`?
11. Are global variables shared between parent and child after `fork()`? Justify with the memory model.
12. Why does removing `sleep()` from only the child in `myfork.c` produce a zombie, and removing it from only the parent produce an orphan?
13. What does `getppid()` return in a child after its real parent has already exited?
14. Why don't file writes from parent and child (same fd, opened pre-fork) overwrite each other?
15. Difference between `exit()` and `_exit()`?
16. What do `WIFEXITED()/WEXITSTATUS()` tell you, vs. `WIFSIGNALED()/WTERMSIG()`?
17. Why must you call `wait()/waitpid()` at all — what resource is reclaimed?

18. Describe the “double fork” daemonizing technique and why it avoids zombies in the original parent.
19. What are the three kernel data structures involved in file sharing, and what does each hold?
20. Why do two separate `open()` calls on the same file get independent file offsets, while a `fork()`-inherited fd doesn’t?
21. In what order do `atexit()`-registered functions run?
22. Explain, using Stevens’ own example, why unflushed `printf()` output before `fork()` can appear twice when redirected to a file but not on a terminal.
23. What is the `TELL_WAIT`/`TELL_PARENT`/`WAIT_CHILD` mechanism used for, and how does it avoid the fork ordering race?
24. Are file locks (`fcntl` locks) inherited across `fork()`?

exec family

25. List the six `exec` functions and explain the naming convention (l/v, p, e).
26. Which of the six is the actual system call, and which are library wrappers?
27. What happens to the calling process’s PID across `exec()`?
28. Why does code written after a successful `execl()` call never run?
29. What does `execl()` return, and under what condition?
30. What determines whether an open fd survives `exec()`?
31. What’s `FD_CLOEXEC`, and how do you set it?
32. If you `exec()` in the parent instead of the child, what’s the practical consequence?
33. How can a child use an fd it never itself `open()`ed, after `exec()`ing a new program?
34. What happens to a caught (non-default, non-ignored) signal handler across `exec()`? Why?
35. Difference between `execlp()`/`execvp()` and their non-`p` counterparts?
36. Why does `exec()`ing a nonexistent program leave your original program still running?

Signals

37. What is the default disposition of `SIGCHLD`? What happens if you explicitly set it to `SIG_IGN`?
38. Why does POSIX recommend `sigaction()` over `signal()`? What does “unreliable signal” mean historically?
39. Difference between blocking a signal and ignoring it?
40. Name functions that are NOT async-signal-safe, and explain the risk of calling them in a handler.
41. What signal is delivered when you `write()` to a pipe with no readers? What’s the default action?
42. How would you use `SIGCHLD` + a handler to reap children without a blocking `wait()` in your main flow?
43. What is `SA_RESTART` used for, and is it per-signal or global?
44. Are signals queued? What happens if the same signal occurs multiple times while blocked?
45. What are the four cases of the `pid` argument to `kill()`, and what does each mean?
46. What’s the difference between `kill()` and `raise()`?
47. What does `alarm()` return, and why is that return value useful?
48. What does `pause()` do, and what’s the classic race condition when combining it with `alarm()`?
49. How does the `sigsetjmp`/`siglongjmp` fix for the alarm/pause race work?
50. What is a “slow” system call, and give three examples?
51. Why does an interrupted slow system call return `-1` with `errno == EINTR`, and what’s the standard retry idiom?
52. What does `sigsuspend()` do, and why must the mask-replace-and-block step be atomic?
53. What was `SIGCLD`, and why did POSIX replace it with `SIGCHLD`?

Pipes

54. What do `fd[0]` and `fd[1]` represent after `pipe()`?
55. Why must `pipe()` be called *before* `fork()` for IPC between parent and child?
56. Why is a pipe called "half-duplex"?
57. Why should each process close the pipe end it doesn't use?
58. What happens if a reader never closes its own write end of the pipe?
59. What happens if you `write()` to a pipe whose read end is fully closed?
60. How do you build a full-duplex channel out of pipes?
61. Describe a scenario where both processes calling `read()` first leads to deadlock.
62. What is `PIPE_BUF`, and what guarantee does it give you?
63. What does `read()` return when all write ends of a pipe are closed and the buffer is empty?

FIFOs

64. How does a FIFO differ from a pipe in naming and process-relationship requirements?
65. What is the blocking/rendezvous behavior of `open()` on a FIFO for O_RDONLY vs O_WRONLY?
66. Why does `fifo.c` open `FIFO1`/`FIFO2` in "crossed" order between parent and child?
67. Does a FIFO's directory entry disappear automatically when the last process closes it?
68. What does `mkfifo()` return if the FIFO already exists, and how does the sample code handle that?
69. Why might FIFOs behave unreliably over NFS?
70. What happens with `O_NONBLOCK` on a FIFO open, for the read side vs. the write side?

Mutexes / condition variables

71. What does `pthread_mutex_lock()` do if another thread already holds the lock?
72. Why must `pthread_cond_wait()` atomically unlock the mutex and block?
73. Why should `cond_wait()` always be called inside a `while` loop, never an `if`?
74. Difference between `pthread_cond_signal()` and `pthread_cond_broadcast()`?

Study tip: for every theory question above, also be ready to sketch the handful of lines of C that demonstrate it — and for every code snippet you're handed on the exam, first ask "what's guaranteed by blocking/waiting here, and what's left to scheduler luck" before writing down an answer.